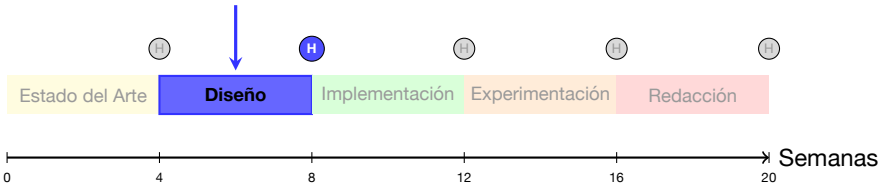
Documento hito #3 (de 6) de mes 2



Diseño Técnico del Algoritmo Genético (GA-Wrapper)

TFM: Selección Automática de Características en Algoritmos de Clasificación Mediante Algoritmos Genéticos y Métodos de Envoltorio con Apache Spark RDD

---

1. Codificación Binaria del Cromosoma

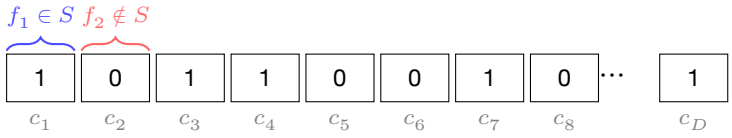
El GA opera sobre una población* de P individuos. Cada individuo es un cromosoma **c** que representa un subconjunto candidato de características del conjunto de datos mediante un vector binario de longitud fija D, donde D es el número total de características:

$$\mathbf{c} = [c_1, c_2, \dots, c_D], \quad c_j \in \{0, 1\}$$

Cada gen c_j codifica la presencia o ausencia de la característica f_j:

- $c_j = 1 \rightarrow$ la característica f_j está **incluida** en el subconjunto S
- $c_j = 0 \rightarrow$ la característica f_j está **excluida** del subconjunto S

El subconjunto activo se define como $S = \{f_j \mid c_j = 1\}$, con cardinalidad $|S|$. El espacio de búsqueda es combinatorio: para un conjunto de datos de D características existen 2^D subconjuntos posibles, lo que hace inviable la búsqueda exhaustiva y justifica el uso de un algoritmo genético.



Cromosoma **c** de longitud D

La longitud del cromosoma y el tamaño del espacio de búsqueda varían según el dataset:

Dataset	D (longitud del cromosoma)	Espacio de búsqueda
WDBC	30	$2^{30} \approx 10^9$ subconjuntos
Arrhythmia	279	2^{279} subconjuntos
Gisette	5,000	2^{5000} subconjuntos

Inicialización de la población: cada gen c_j se inicializa de forma independiente con probabilidad p_{init}. Se usa $p_{\text{init}} = 0.5$ (uniforme) para WDBC y Arrhythmia, lo que garantiza diversidad inicial. Para Gisette se aplica una inicialización dispersa ($p_{\text{init}} = 0.1$) con el fin de evitar partir de cromosomas con más de 2500 genes activos, lo que colapsaría el coste computacional ya en la primera generación (Siedlecki & Sklansky, 1989).

2. Función de Fitness

La función de fitness implementa un criterio multiobjetivo del método wrapper, que equilibra precisión del clasificador y parsimonia del subconjunto:

$$\text{Fitness}(\mathbf{c}) = \alpha \cdot \text{Accuracy}(\mathbf{c}) - \beta \cdot \frac{|S|}{D}$$

donde:

- Accuracy$(\mathbf{c})$ es la exactitud media obtenida mediante validación cruzada de 5 pliegues (*5-fold cross-validation*) con el clasificador base sobre las N instancias, usando únicamente las características en S;
- $|S|/D$ es la proporción normalizada de características seleccionadas (penalización por complejidad);
- $\alpha = 0.9 \rightarrow$ peso de la precisión (objetivo principal);
- $\beta = 0.1 \rightarrow$ peso de la penalización por número de características.

Los valores $\alpha = 0.9$ y $\beta = 0.1$ son los recomendados como punto de partida en la literatura de GA-wrapper (Kohavi & John, 1997; Siedlecki & Sklansky, 1989) y serán objeto de análisis de sensibilidad durante la fase de experimentación. El fitness máximo teórico es 1.0 (exactitud del 100% con una sola característica).

Como caso especial, si $|S| = 0$ (cromosoma nulo), el fitness se fija a 0 para penalizar soluciones inviables.

3. Operadores Genéticos

Los operadores genéticos se implementan con DEAP (*Distributed Evolutionary Algorithms in Python*, Fortin et al., 2012), librería de referencia para algoritmos evolutivos en Python. Se escoge frente a alternativas como `pygad` o `geneticalgorithm` por su soporte nativo de cromosomas binarios, sus operadores modulares intercambiables y su compatibilidad directa con `RDD.map()` de Spark, ya que la función de fitness está escrita en Python puro y no requiere modificaciones para ser distribuida.

Librería	Selección/descarte
DEAP (Sí)	Operadores binarios nativos. Fitness como función Python pura $\rightarrow$ compatible con `RDD.map()`. >4,000 citas (JMLR 2012).
`pygad` (No)	Representación continua por defecto; adaptación binaria compleja.
`geneticalgorithm` (No)	Operadores fijos; integración con Spark inviable.
`pyevolve` (No)	Sin mantenimiento desde 2012.

3.1 Selección: Torneo Binario

Se emplea selección por torneo de tamaño 2 (`tools.selTournament`, DEAP):

- Se seleccionan aleatoriamente 2 individuos de la población;
- Avanza el de mayor fitness;
- El proceso se repite hasta completar la población de P padres.

Este tipo de torneo binario se justifica porque equilibra presión selectiva y diversidad; es más robusto que la selección por ruleta ante valores de fitness similares (Goldberg, 1989, cap. 2) y compatible con poblaciones paralelas en Spark.

3.2 Cruce: Un Punto (Single-Point Crossover)

Se aplica cruce de un punto (`tools.cx0nePoint`, DEAP) con probabilidad p_{cross}:

- se selecciona un punto de corte aleatorio $k \in [1, D-1]$;
- hijo 1: $\mathbf{c}_1[1{:}k] \,\|\, \mathbf{c}_2[k{:}D]$;
- hijo 2: $\mathbf{c}_2[1{:}k] \,\|\, \mathbf{c}_1[k{:}D]$.

Para Gisette ($D = 5000$) se evaluará adicionalmente el cruce uniforme (`cxUniform`, $p = 0.5$), que ofrece mayor diversidad en espacios de alta dimensionalidad (Mitchell, 1996).

3.3 Mutación: Flip de Bit

Se aplica mutación por inversión de bit (`tools.mutFlipBit`, DEAP) con probabilidad por gen $p_{\text{mut_bit}}$:

- cada bit c_j se invierte con probabilidad independiente $p_{\text{mut_bit}}$;
- el operador permite tanto activar características no seleccionadas como eliminar características ya seleccionadas.

La probabilidad individual $p_{\text{mut_bit}}$ se escala inversamente con D para mantener un número esperado constante de bits mutados:

$$p_{\text{mut_bit}} = \frac{1}{D} \quad \Rightarrow \quad \mathbb{E}[\text{bits mutados}] \approx 1 \text{ por cromosoma}$$

4. Esquema del Ciclo Evolutivo (Pseudocódigo)

```
INICIALIZAR población P_0 de tamaño P con vectores binarios aleatorios
DISTRIBUIR P_0 como RDD en Spark (paralelización de evaluación de fitness)

PARA g = 1 hasta G:
    # Evaluación paralela (Spark RDD)
    fitness_scores = RDD(P_g).map(evaluar_fitness).collect()

    # Elitismo: preservar top-E individuos
    elite = seleccionar_top_E(P_g, fitness_scores, E)

    # Selección por torneo
    padres = torneo_binario(P_g, fitness_scores)

    # Cruce (p_cross) y Mutación (p_mut, p_mut_bit)
    descendencia = aplicar_cruce_y_mutacion(padres)

    # Nueva generación
    P_{g+1} = elite + descendencia[:P-E]

    # Verificar criterio de parada
    SI convergencia(fitness_scores, g): BREAK

RETORNAR mejor_individuo -> S* = {f_j | c_j = 1}
```

Al finalizar, el subconjunto óptimo seleccionado es $S^* = \{f_j \mid c_j = 1\}$.

5. Clasificador Base para el Wrapper

5.1 Clasificador primario durante la búsqueda: Naive Bayes gaussiano (GNB)

El clasificador base empleado para calcular Accuracy$(\mathbf{c})$ durante la evolución del GA es el Naive Bayes gaussiano, implementado con `scikit-learn` (`GaussianNB`).

Justificación:

Criterio	Justificación
Velocidad $\mathcal{O}(N \times D)$	Cada ejecución del GA genera $P \times G \times k$ entrenamientos. Con $P = 100$, $G = 100$, $k = 5$ $\rightarrow$ 50 000 entrenamientos por ejecución. El GNB es el único clasificador viable en este escenario.
Alta dimensionalidad	No padece la maldición de la dimensionalidad gracias a la suposición de independencia condicional. Es estable con subconjuntos dispersos en las primeras generaciones.
Sin hiperparámetros	Elimina la variabilidad experimental, ya que no requiere ajuste interno durante la búsqueda.
Validado en la literatura	Empleado como clasificador base en selección de características con GA-wrapper (Aalaei et al., 2016; Huang et al., 2020; Rashid & Abdullah, 2014).

5.2 Clasificador secundario para validación: SVM con núcleo lineal

Una vez convergido el GA, el subconjunto S^* se revalúa con `LinearSVC` para confirmar la generalización de la selección y medir la aceleración (*speed-up*) de Spark bajo un mayor coste computacional por evaluación (véase el documento de delimitación del problema de clasificación, §4.5).

6. Parámetros Iniciales del GA por Dataset

Los parámetros se fijan de forma diferenciada según la dimensionalidad y el coste computacional de cada conjunto de datos, siguiendo las recomendaciones de Goldberg (1989) y los precedentes en la literatura de GA-wrapper:

Parámetro	WDBC ($D = 30$)	Arrhythmia ($D = 279$)	Gisette ($D = 5000$)
Tamaño población (P)	50	100	100
Generaciones (G)	50	100	100
p_{cross} **(cruce)**	0.8	0.8	0.8
p_{mut} **(mutación individuo)**	0.2	0.2	0.2
$p_{\text{mut_bit}}$ **(mutación por gen)**	$1/30 \approx 0.033$	$1/279 \approx 0.004$	$1/5000 = 0.0002$
Folds CV (k)	5	5	5
Elitismo	Top 2	Top 5	Top 5
p_{init} **(inicialización)**	0.5	0.5	0.1
Criterio de parada	$G = 50$ o Δfitness < 0.001 (10 gen)	$G = 100$ o Δfitness < 0.001 (15 gen)	$G = 100$ o Δfitness < 0.001 (15 gen)
Entrenamientos estimados ($P \times G \times k$)	12,500	50,000	50,000

Elitismo: en cada generación, los E mejores individuos (por fitness) pasan directamente a la siguiente generación sin cruce ni mutación. Esto garantiza la monotonía del mejor fitness y evita la pérdida de buenas soluciones (Goldberg, 1989).

Criterio de parada dual: el GA termina cuando se alcanza el número máximo de generaciones G, o cuando la mejora del mejor fitness en las últimas N_{stop} generaciones es inferior a 0.001. Esto evita iteraciones innecesarias una vez que el algoritmo ha convergido.

Referencias

- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley. Cap. 1-3.
- Mitchell, M. (1996). *An Introduction to Genetic Algorithms*. MIT Press.
- Kohavi, R., & John, G. H. (1997). Wrappers for feature subset selection. *Artificial Intelligence*, 97(1-2), 273-324. DOI: 10.1016/S0004-3702(97)00043-X
- Siedlecki, W., & Sklansky, J. (1989). A note on genetic algorithms for large-scale feature selection. *Pattern Recognition Letters*, 10(5), 335-347. DOI: 10.1016/0167-8655(89)90037-8
- Aalaei, S., et al. (2016). Feature selection using genetic algorithm for breast cancer diagnosis. *Journal of Biomedical Informatics*, 60, 1-7.
- Huang, J., et al. (2020). GARS: Genetic Algorithm for the identification of a Robust Subset of features. *BMC Bioinformatics*, 21(54).
- Fortin, F. A., et al. (2012). DEAP: Evolutionary Algorithms Made Easy. *JMLR*, 13, 2171-2175. https://deap.readthedocs.io/